

```

// Headless verification of the demo's threading math (mirrors the pure
// functions in 25_tetra_threading_explorer.html)
const Zv = 150, Zh = 250;
const rayAt = (P,z)=>[P[0]*(z-Zv)/(P[2]-Zv), P[1]*(z-Zh)/(P[2]-Zh), z];
const foot1=(P)=>rayAt(P,Zv), foot2=(P)=>rayAt(P,Zh), scr=(P)=>rayAt(P,0);
const collin=(P)=>{const A=foot1(P),B=foot2(P);
  const u=[B[0]-A[0],B[1]-A[1],B[2]-A[2]],v=[P[0]-A[0],P[1]-A[1],P[2]-A[2]];
  const c=[u[1]*v[2]-u[2]*v[1],u[2]*v[0]-u[0]*v[2],u[0]*v[1]-u[1]*v[0]];
  return Math.hypot(...c)/(Math.hypot(...u)*Math.hypot(...v));};

let worst=0, worstFoot=0;
for(let i=0;i<20000;i++){
  const z = 30+Math.random()*410;
  if(Math.abs(z-Zv)<3||Math.abs(z-Zh)<3) continue;
  const P=[(Math.random()-0.5)*600,(Math.random()-0.5)*600,z];
  worst=Math.max(worst, collin(P)); // Thm 1: P on ray
  worstFoot=Math.max(worstFoot,
    Math.abs(foot1(P)[0]-0), Math.abs(foot1(P)[2]-Zv), // F1 on l1
    Math.abs(foot2(P)[1]-0), Math.abs(foot2(P)[2]-Zh)); // F2 on l2
}
console.log("collinearity worst-case :", worst.toExponential(2));
console.log("feet-on-slit worst-case :", worstFoot.toExponential(2));

// Rig sanity vs Theorem 7 example: z=350 -> Mx/My = 0.3
const z=350, Mx=Zv/(z-Zv), My=Zh/(z-Zh);
console.log("AR at z=350 (expect 0.3) :", (Mx/My).toFixed(12));

// Uniqueness cross-check: reconstruct P from (s,t,z) and compare
const P=[60,45,350], s=foot1(P)[1], t=foot2(P)[0];
// ray through (0,s,Zv) and (t,0,Zh), evaluated at z=350:
const lam=(350-Zv)/(Zh-Zv);
const Q=[0+t*lam, s+(0-s)*lam, 350];
console.log("round-trip P vs (s,t) ray :", Math.hypot(Q[0]-P[0],Q[1]-P[1]).toExpon

```